

Tema 2 - CSS y construcción de interfaces web

2.5 - Componentes HTML nativos y controles de interfaz

HTML proporciona el control; CSS define su presentación

Una interfaz web contiene numerosos elementos con los que se puede interactuar: campos de entrada, casillas de selección, controles deslizantes, barras de progreso, desplegables o cuadros de diálogo. Aunque sería posible reconstruir muchos de ellos mediante elementos genéricos, HTML ya proporciona controles diseñados específicamente para esas funciones. Utilizar un elemento nativo permite partir de una estructura cuyo significado y parte de cuyo comportamiento ya conoce el navegador. CSS puede intervenir después para integrarlo visualmente con el resto de la interfaz. Por ejemplo, un bloque compuesto exclusivamente por elementos genéricos no adquiere automáticamente comportamiento desplegable:

Elementos genéricos

```
<div class="pregunta">
  <div class="titulo">
    Requisitos del curso
  </div>
  <div class="respuesta">
    Conocimientos básicos de HTML.
  </div>
</div>
```

Elemento nativo

```
<details>
  <summary>Requisitos del
curso</summary>
  <p>Conocimientos básicos de HTML.
</p>
</details>
```

En el segundo caso, el navegador ya conoce el estado abierto o cerrado y permite interactuar con el componente sin necesidad de reconstruir ese funcionamiento mediante CSS.

CSS conserva, sin embargo, un papel fundamental. Puede controlar aspectos como colores, bordes, separación, dimensiones, tipografía, estados, indicadores, foco e integración con la guía visual del sitio.



La misma idea se aplica a numerosos controles de formulario. Un checkbox sigue siendo funcional sin CSS, pero su presentación puede adaptarse a la identidad visual de la aplicación. Un `<progress>` ya representa progreso, pero CSS puede ayudar a integrarlo con el resto de los componentes.

> Cuando HTML proporciona un elemento adecuado para una función, resulta preferible partir de ese elemento y utilizar CSS para su presentación antes que reconstruir el componente desde elementos genéricos.

`<details>` y `<summary>`: contenido desplegable nativo

`<details>` permite crear un bloque cuyo contenido adicional puede permanecer oculto hasta que se solicite. `<summary>` proporciona el resumen o título visible que controla la apertura y cierre del componente. Este patrón resulta útil para información secundaria que no necesita estar permanentemente desplegada: requisitos, explicaciones adicionales, preguntas frecuentes o detalles sobre un recurso.

```
<details>
  <summary>
    ¿Qué conocimientos previos necesito?
  </summary>
  <p>
    Es conveniente conocer la estructura básica de un documento HTML
    y los fundamentos de CSS.
  </p>
</details>
```

El contenido de `<summary>` permanece visible en ambos estados. El resto del contenido de `<details>` se muestra cuando el componente está abierto. `<summary>` debe ocupar la primera posición dentro del `<details>` cuando se utiliza como etiqueta del componente.



El contenido desplegado no está limitado a texto simple:

```
<details>
  <summary>Contenido del curso</summary>
  <p>El curso incluye:</p>
  <ul>
    <li>Fundamentos de CSS</li>
    <li>Construcción de interfaces</li>
    <li>Diseño responsive</li>
  </ul>
</details>
```

El navegador proporciona directamente la interacción de apertura y cierre. Por tanto, CSS puede centrarse en la presentación del componente.

Presentación y estados de `<details>` y `<summary>`

El atributo `open`

El atributo booleano `open` representa el estado abierto de `<details>`.

Al ser un atributo booleano, lo importante es su presencia. Por ello, `<details open="false">` también representa un elemento abierto. Para indicar el estado cerrado, el atributo debe estar ausente.

```
<details open>
  <summary>Información importante</summary>
  <p>
    Este contenido aparece desplegado inicialmente.
  </p>
</details>
```

CSS puede utilizar este estado mediante un selector de atributo:

```
details {
  padding: 1rem;
  border: 1px solid #cbd5e1;
  border-radius: 0.75rem;
}
details[open] {
  border-color: #2266cc;
  background-color: #eff6ff;
}
```

El selector `details[open]` solo coincide cuando el atributo `open` está presente. De este modo puede modificarse visualmente el componente según su estado sin añadir una clase específica para indicar si está abierto.

Presentar el resumen

```
summary {
  color: #172554;
  font-weight: 700;
  cursor: pointer;
}
summary::marker {
  color: #7c3aed;
}
```

`<summary>` suele representarse como un elemento de lista con un marcador que indica que puede desplegarse.

`::marker`, estudiado anteriormente para listas, también permite intervenir sobre el marcador asociado al `<summary>`. No siempre es necesario eliminar el indicador nativo. Un marcador reconocible ayuda a comunicar que el elemento puede abrirse.

Presentación y estados de `<details>` y `<summary>`

Varios bloques relacionados

Sin una relación adicional, varios `<details>` pueden permanecer abiertos simultáneamente. HTML también permite agruparlos utilizando un mismo `name`:

```
<details name="temas">
  <summary>HTML</summary>
  <p>Estructura del documento.</p>
</details>
<details name="temas">
  <summary>CSS</summary>
  <p>Presentación de la interfaz.</p>
</details>
<details name="temas">
  <summary>JavaScript</summary>
  <p>Comportamiento programado.</p>
</details>
```

Los elementos que comparten el mismo `name` forman un grupo exclusivo. Cuando se abre uno, se cierra el que estuviera abierto dentro del mismo grupo. Esto permite construir determinados comportamientos de tipo acordeón sin añadir programación.

La agrupación mediante `name` es una incorporación más reciente que el funcionamiento básico de `<details>`, por lo que conviene comprobar la compatibilidad cuando sea necesario soportar navegadores antiguos.



`<dialog>`: estructura nativa y presentación con CSS

HTML proporciona `<dialog>` para representar cuadros de diálogo y otros componentes temporales relacionados con la interacción. Su utilización permite disponer de un elemento cuyo propósito está definido por HTML en lugar de representar el mismo concepto mediante un `<div>` genérico.

En este apartado interesa principalmente su **presentación mediante CSS**. La programación utilizada por una aplicación para abrir, cerrar o controlar dinámicamente un diálogo pertenece al comportamiento de la interfaz y queda fuera de este desarrollo.

Para observar su estructura puede utilizarse temporalmente el atributo `open`:

```
<dialog open class="dialogo-curso">
  <h2>Información del curso</h2>
  <p>La matrícula permanecerá abierta hasta completar las plazas.</p>
  <button type="button">Cerrar</button>
</dialog>
```

El elemento puede recibir estilos como cualquier otro componente:

```
.dialogo-curso {
  max-width: 32rem;
  padding: 1.5rem;
  color: #172554;
  background-color: white;
  border: 0;
  border-radius: 1rem;
  box-shadow: 0 1rem 2.5rem rgb(15 23 42 / 0.18);
}
```

No es necesario modificar su estructura HTML para controlar aspectos como tamaño, fondo, borde, tipografía o sombra.

`<dialog>`: estructura nativa y presentación con CSS

La superficie situada detrás de un diálogo modal

CSS proporciona el pseudoelemento `::backdrop` para presentar la superficie situada detrás de determinados elementos cuando se encuentran en la capa superior del navegador.

```
dialog::backdrop {
   background-color: rgb(15 23 42 / 0.65);
}
```

Esta regla permite oscurecer el contenido situado detrás de un diálogo cuando este se muestra modalmente.

CSS puede definir la apariencia del diálogo y de `::backdrop`.

La lógica que abre un diálogo dinámicamente como modal no pertenece a CSS.

La API utilizada para controlar programáticamente `<dialog>` no se desarrolla en este apartado.



`<datalist>` y `<select>`: sugerir o limitar opciones

No todos los controles que presentan opciones cumplen la misma función. En algunos casos la elección debe limitarse a una lista cerrada. En otros interesa proporcionar sugerencias sin impedir que se escriba un valor diferente. `<datalist>` permite asociar sugerencias a determinados elementos `<input>`:

```
<label for="tecnologia">Tecnología principal</label>
<input type="text" id="tecnologia" name="tecnologia" list="tecnologias">
<datalist id="tecnologias">
  <option value="HTML">
  <option value="CSS">
  <option value="JavaScript">
  <option value="PHP">
</datalist>
```

El atributo `list` del `<input>` contiene el identificador del `<datalist>` correspondiente. El navegador puede mostrar los valores definidos mediante `<option>` como sugerencias. Sin embargo, continúa siendo posible introducir otro valor.

Esto lo diferencia de `<select>`:

```
<label
  for="tecnologia-principal">
  Tecnología principal
</label>
<select
  id="tecnologia-principal"
  name="tecnologia-principal">
  <option>HTML</option>
  <option>CSS</option>
  <option>JavaScript</option>
  <option>PHP</option>
</select>
```



No todos los controles permiten el mismo nivel de personalización

Una diferencia importante entre elementos HTML ordinarios y determinados controles nativos es que parte de su interfaz puede estar proporcionada por el navegador o por el sistema operativo. La lista desplegada por un `<datalist>`, por ejemplo, ofrece posibilidades de personalización visual mucho más limitadas que un bloque HTML normal. Esto conduce a una decisión habitual en diseño de interfaces: **no todos los detalles visuales necesitan ser idénticos si para conseguirlo habría que reconstruir innecesariamente un control que el navegador ya resuelve correctamente**.

`<progress>` y `<meter>`: barras con significados diferentes

`<progress>` y `<meter>` pueden representarse visualmente mediante formas parecidas, pero describen situaciones distintas. Elegir entre ambos únicamente por su apariencia produciría una estructura HTML incorrecta.

`<progress>`: avance de una tarea

`<progress>` representa el progreso de una operación que todavía no ha finalizado.

```
<label for="descarga">
  Descarga del material
</label>
<progress
  id="descarga"
  value="65"
  max="100"
>
  65 %
</progress>
```

`value` representa la cantidad completada y `max` establece el valor correspondiente a la finalización.

También puede indicarse que existe una tarea en curso sin conocer todavía su grado de avance:

```
<label for="procesando">
  Preparando archivo
</label>
<progress id="procesando">
  Procesando…
</progress>
```

Al no existir `value`, el progreso es indeterminado.

`<meter>`: medición dentro de un intervalo

`<meter>` representa un valor dentro de un rango conocido.

```
<label for="ocupacion">
  Ocupación
</label>
<meter
  id="ocupacion"
  min="0"
  max="100"
  low="40"
  high="80"
  optimum="20"
  value="72"
>
  72 %
</meter>
```

Sus atributos permiten describir el intervalo y la posición del valor:

- `min`: límite mínimo;
- `max`: límite máximo;
- `value`: valor actual;
- `low`: límite de la zona baja;
- `high`: límite de la zona alta;
- `optimum`: valor o intervalo considerado preferible.



`<output>`: identificar y presentar un resultado

`<output>` representa un resultado producido a partir de una operación o interacción. Su función dentro de HTML es identificar ese contenido como resultado, no realizar el cálculo que lo genera. Por ejemplo:

```
<div class="resultado-curso">
  <span>Duración semanal</span>
  <output>4 horas</output>
</div>
```

Puede presentarse mediante CSS como cualquier otro contenido:

```
.resultado-curso output {
  display: inline-block;
  padding: 0.4rem 0.75rem;
  color: #172554;
  background-color: #eff6ff;
  border: 1px solid #bfdbfe;
  border-radius: 0.5rem;
  font-weight: 700;
}
```



`<output>` también dispone del atributo `for`, que puede relacionarlo con los identificadores de controles que intervienen en la obtención del resultado:

```
<output for="precio unidades">
   48 €
</output>
```

Esto no significa que el navegador multiplique automáticamente los valores de precio y unidades.

HTML

Identifica el contenido como resultado.

CSS

Controla su presentación.

Lógica de la aplicación

Obtiene o actualiza el resultado cuando sea necesario.

La tercera parte no se desarrolla en este apartado. Un `<span>` podría mostrar visualmente el texto 48 €, pero `<output>` aporta significado a ese contenido cuando realmente representa el resultado de una operación.

Tipos de `<input>`: un elemento para diferentes clases de datos

`<input>` representa un campo de datos cuyo comportamiento depende en gran medida del atributo `type`. No todos los valores generan un control visualmente distinto, pero cada tipo comunica al navegador qué clase de información se espera y puede activar comportamientos específicos.

Familia	type	Finalidad principal
Texto	text	Texto de una línea
Texto	search	Términos de búsqueda
Texto	email	Dirección de correo electrónico
Texto	password	Información textual que debe ocultarse visualmente
Fecha y hora	date	Fecha
Numéricos	number	Valor numérico
Numéricos	range	Valor dentro de un intervalo cuando el valor exacto no es el aspecto principal
Visual	color	Selección de color
Selección	checkbox	Selección independiente
Selección	radio	Una opción dentro de un grupo
Archivo	file	Selección de uno o varios archivos
Acción	submit	Envío de formulario
Acción	reset	Restablecimiento de valores
Acción	button	Botón sin comportamiento de envío predeterminado
No visible	hidden	Valor sin control visual

HTML define además otros tipos específicos, como `tel`, `url`, `month`, `week`, `time`, `datetime-local`, `reset` e `image`, que pueden utilizarse cuando la aplicación necesita representar esos datos o acciones concretas.

Si `type` se omite o se proporciona un valor no reconocido, el estado predeterminado es `text`.

<input> · TIPOS DE CONTROL

El atributo **type** define el dato o la acción.

TEXTO Y DATOS (6)

text
Texto en una sola línea.

search
Texto para una búsqueda.

email
Dirección de correo electrónico.
ana@aula.es

tel
Número de teléfono.

url
Dirección web.
https://aula.es

password
Texto con caracteres ocultos.
•••••••

FECHA Y HORA (5)

date
Fecha: año, mes y día.
2024-06-24

month
Mes y año.
2024-06

week
Semana y año.
2024-W26

time
Hora, sin fecha.
14:30

datetime-local
Fecha y hora, sin zona horaria.
2024-06-24T14:30

VALORES (3)

number
Valor numérico.
10

range
Valor dentro de un intervalo.

color
Selección de un color.

SELECCIÓN (2)

checkbox
Opción que se marca de forma independiente.

radio
Una opción de un mismo grupo.

ARCHIVOS (1)

file
Selección de uno o varios archivos.
Seleccionar archivos
Ningún archivo seleccionado.

ACCIONES (4)

submit
Envía el formulario.

reset
Restablece los valores iniciales.

button
Botón de acción programable.

image
Botón de envío mediante una imagen.

NO VISIBLE (1)

hidden
Valor enviado sin control visible.
a7f3c9

Si type se omite o no se reconoce → text

Presentación común de campos de texto, búsqueda, correo y número

Varios tipos de `<input>` se presentan mediante campos con una estructura visual semejante. Esto permite establecer una base común mediante CSS y aplicar posteriormente las diferencias que sean necesarias.

```
<label for="nombre">Nombre</label>
<input type="text" id="nombre">

<label for="buscar">Buscar</label>
<input type="search" id="buscar">

<label for="correo">Correo</label>
<input type="email" id="correo">

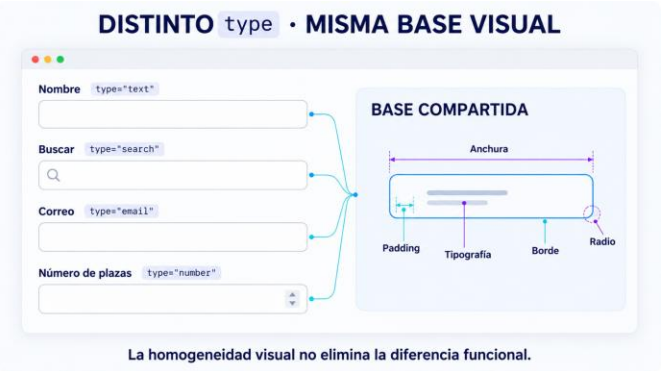
<label for="plazas">Número de plazas</label>
<input type="number" id="plazas">
```

Una presentación común puede establecerse de forma conjunta:

```
input[type="text"],
input[type="search"],
input[type="email"],
input[type="number"] {
  width: 100%;
  padding: 0.75rem 0.9rem;
  color: #172554;
  background-color: white;
  border: 1px solid #cbd5e1;
  border-radius: 0.5rem;
  font: inherit;
}
```

Los selectores de atributo permiten aplicar estilos según el valor de `type`. También puede existir una diferencia concreta:

```
input[type="search"] {
  background-color: #f8fafc;
}
```



El objetivo no es crear una regla independiente para cada tipo si todos comparten la misma presentación. Agrupar selectores evita duplicar declaraciones y mantiene una base visual coherente.

El control sigue conservando su tipo

Que varios campos compartan colores, bordes o tipografía no elimina sus diferencias funcionales. Un campo `<input type="email">` continúa representando un correo electrónico aunque CSS haga que su caja tenga un aspecto similar a `<input type="text">`.

La homogeneidad visual no exige eliminar las diferencias semánticas y funcionales de los controles.

`range`, `color` y `date`: controles proporcionados por el navegador

Algunos tipos de `<input>` generan controles cuya interfaz depende en mayor medida del navegador y del sistema operativo. CSS permite integrarlos parcialmente, pero no siempre proporciona el mismo nivel de control que sobre una caja HTML ordinaria.

`range`

`range` permite seleccionar un valor dentro de un intervalo:

```
<label for="volumen">
  Volumen
</label>
<input
  type="range"
  id="volumen"
  min="0"
  max="100"
  step="5"
  value="60"
>
```

Los principales atributos del ejemplo son:

- `min`: valor mínimo;
- `max`: valor máximo;
- `step`: incremento;
- `value`: valor inicial.

No debe confundirse con `<progress>`. `range` permite introducir un valor; `<progress>` representa el avance de una operación.

`color`

```
<label for="color-acento">
  Color de acento
</label>
<input
  type="color"
  id="color-acento"
  value="#2266cc"
>
```

El navegador proporciona el mecanismo de selección de color adecuado para la plataforma.

`date`

```
<label for="fecha-inicio">
  Fecha de inicio
</label>
<input
  type="date"
  id="fecha-inicio"
  min="2026-09-01"
  max="2027-06-30"
>
```

El navegador puede proporcionar un selector específico de fecha y utilizar los límites definidos mediante `min` y `max`.

Presentación nativa y sistema operativo

El aspecto exacto de estos controles puede cambiar entre navegadores, sistemas operativos y dispositivos. No debe considerarse un error que un selector de fecha o de color no sea idéntico en dos plataformas. En componentes nativos, mantener su comportamiento y usabilidad puede ser más importante que intentar conseguir una reproducción visual idéntica en todos los navegadores.

Checkbox y radio

Checkbox y radio representan dos formas diferentes de selección.

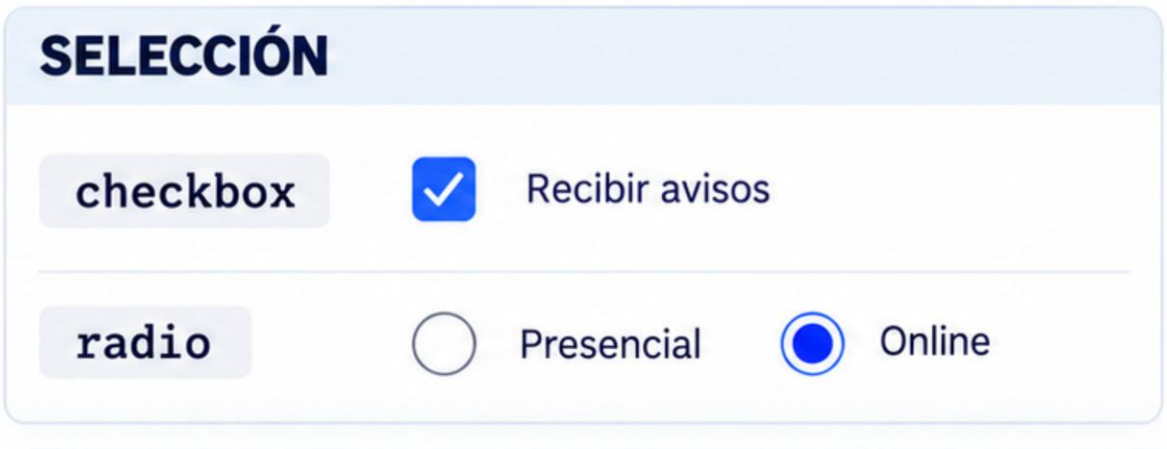
Un checkbox permite activar o desactivar una opción de manera independiente:

```
<label>
  <input type="checkbox" name="avisos" checked>
  Recibir avisos
</label>
```

Los botones radio se utilizan cuando varias opciones pertenecen al mismo grupo y se desea seleccionar una sola:

```
<label>
  <input type="radio" name="modalidad" value="presencial">
  Presencial
</label>
<label>
  <input type="radio" name="modalidad" value="online" checked>
  Online
</label>
```

El mismo valor de `name` relaciona los botones radio como alternativas de un único grupo.



`accent-color`

Adaptar el color de acento

CSS proporciona `accent-color` para modificar el color de acento utilizado por determinados controles generados por el navegador.

```
:root {
  --color-acento: #7c3aed;
}
input {
  accent-color: var(--color-acento);
}
```

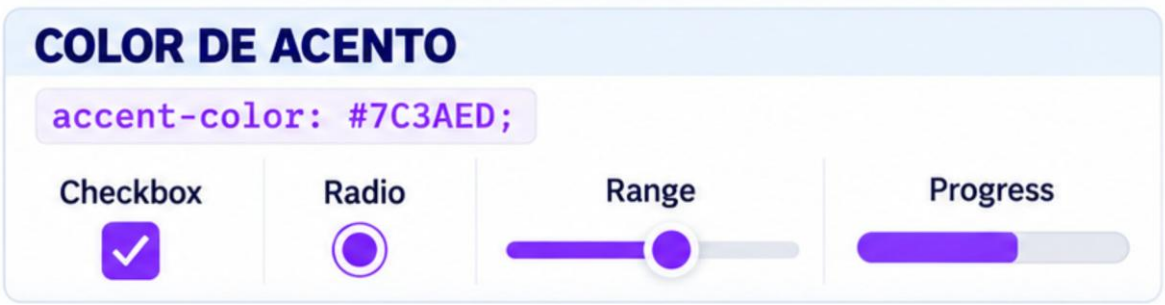
La propiedad puede afectar, en los navegadores compatibles, a controles como checkbox, radio, range y progress.

```
input[type="checkbox"],
input[type="radio"],
input[type="range"],
progress {
  accent-color: var(--color-acento);
}
```

`accent-color` no sustituye a `color`, `background-color`, `border-color` u otras propiedades. Su objetivo es indicar el color de acento que puede utilizar el navegador en los controles compatibles.

Tampoco implica que todos los detalles del control utilicen exactamente el color especificado. El navegador conserva parte de su representación.

En 2026, `accent-color` sigue presentando diferencias de disponibilidad entre navegadores, por lo que resulta apropiado utilizarla como **mejora progresiva**: si no se aplica, el control mantiene su aspecto nativo y continúa siendo funcional.



Pseudoclases de estado para controles de interfaz

HTML ya conoce numerosos estados de los controles: si una casilla está marcada, si un elemento está deshabilitado, si un dato es obligatorio o si el contenido satisface determinadas restricciones. CSS puede utilizar esos estados mediante pseudoclases sin necesidad de duplicarlos mediante clases añadidas manualmente.

`:checked` - Selecciona checkbox y radio que se encuentran marcados:

```
input:checked { accent-color: #2266cc; }
```

`:disabled` - Selecciona controles deshabilitados:

```
<button type="button" disabled> Entregar actividad </button>

button:disabled { opacity: 0.45; cursor: not-allowed; }
```

HTML establece que el control está deshabilitado. CSS únicamente representa visualmente ese estado.

`:required` - Selecciona controles que incluyen la restricción `required`:

```
<input type="email" required>

input:required { border-inline-start: 0.25rem solid #2266cc; }
```

`:valid` e `:invalid` - Determinados controles pueden compararse con las restricciones definidas en HTML:

```
input:valid { border-color: #15803d; }
input:invalid { border-color: #b91c1c; }
```

Por ejemplo, un input obligatorio de tipo email puede estar en estado inválido cuando no contiene un valor que satisfaga sus restricciones. Debe tenerse en cuenta que `:invalid` puede coincidir antes de que se haya intentado completar el formulario. Por ello, no siempre resulta conveniente mostrar desde el primer instante una presentación de error excesivamente intensa.

> Las pseudoclases de estado permiten que CSS responda a información que HTML y el navegador ya conocen. No resulta necesario mantener manualmente clases como `.marcado`, `.desactivado`, `.obligatorio` o `.invalido` cuando únicamente duplicarían un estado nativo del control.

Foco, placeholder y presentación coherente de un formulario

Además de estados relacionados con el valor del control, CSS puede responder a situaciones importantes durante la interacción.

Foco visible

`:focus-visible` permite aplicar una presentación cuando el navegador determina que debe mostrarse una indicación visible del foco.

```
input:focus-visible,
select:focus-visible,
button:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
```

El indicador de foco no debe eliminarse sin proporcionar una alternativa suficientemente clara. Aunque la accesibilidad se desarrollará específicamente más adelante, una interfaz no debe enseñarse utilizando desde el principio una práctica que dificulte la navegación mediante teclado.

Presentar el placeholder

`::placeholder` permite modificar la apariencia del texto definido mediante el atributo `placeholder`.

```
<label for="buscar">
  Buscar curso
</label>
<input
  type="search"
  id="buscar"
  placeholder="Ej.: CSS"
>

input::placeholder {
  color: #64748b;
  font-style: italic;
}
```

El placeholder proporciona una pista, pero no sustituye a `<label>`. La etiqueta identifica permanentemente el campo; el placeholder desaparece cuando se introduce un valor.

Referencias y recursos de consulta

HTML Living Standard — The input element
WHATWG. Referencia normativa actual sobre `<input>`, sus tipos y comportamiento. Utilizada para comprobar los tipos definidos actualmente por HTML y el comportamiento predeterminado de `type`.
HTML Living Standard · input

<details>: The Details disclosure element
MDN Web Docs. Documentación técnica sobre `<details>`, `<summary>`, `open`, `name` y presentación del componente. **MDN · details**

<summary>: The Disclosure Summary element
MDN Web Docs. Referencia sobre el funcionamiento y presentación de `<summary>` y su marcador.
MDN · summary

<dialog>: The Dialog element
MDN Web Docs. Referencia sobre la estructura del elemento `<dialog>`, el atributo `open` y el pseudoelemento `::backdrop`. **MDN · dialog**

<input>: The Input element
MDN Web Docs. Referencia sobre los diferentes controles de entrada, atributos y pseudoclases asociadas. **MDN · input**

UI pseudo-classes
MDN Web Docs. Referencia sobre pseudoclases de estado aplicables a controles de formulario.
MDN · UI pseudo-classes

accent-color
MDN Web Docs. Referencia sobre la propiedad CSS utilizada para definir el color de acento de determinados controles nativos.
MDN · accent-color

Idea final

Los controles nativos no impiden diseñar una interfaz propia. Proporcionan una base funcional sobre la que CSS puede construir una presentación coherente, evitando sustituir comportamientos que el navegador ya ofrece.